MECHATRONICS AND IOT

ASSIGNMENT REPORT - 4

TITLE:

Design and simulate a Smart Flood Level Monitoring System using Ultrasonic Sensor. The system continuously monitors water levels and provides alerts during critical flood conditions for disaster management applications.

DONE BY ,

HARI KRISHNA JAGADEESH-24MER011

HARISH DHARMALINGAM – 24MER014

RANJITH KUMAR P – 24MER048

RISHI KESAV A – 24MER049

VENKATRAMANAN B – 24MER061

Project Overview:

The Smart Flood Level Monitoring System is an IoT-based system designed to continuously monitor water levels and provide early warnings during potential flood conditions. The system uses an ultrasonic sensor to measure the distance between the sensor and the water surface. A microcontroller processes the measured distance and determines the corresponding water level.

Based on predefined water-level thresholds, the system classifies the condition as Normal, Warning, or Critical. LEDs and a buzzer can be used to provide immediate local alerts when the water level reaches a critical value. The system can also be extended with Wi-Fi connectivity to transmit water-level information to a cloud platform for remote monitoring.

The proposed system provides a low-cost, real-time and automated solution for flood-level monitoring and can be used in rivers, dams, drainage channels, reservoirs and other flood-prone areas to support disaster management and early warning systems.

Problem Statement:

Floods can cause severe damage to human life, property, infrastructure and the environment, particularly in areas located near rivers, dams and drainage systems. Conventional flood monitoring methods may depend on manual observation or periodic measurements, which can result in delayed detection and warning when water levels rise rapidly.

The absence of a continuous and automated monitoring system makes it difficult to provide timely alerts during critical flood conditions. Therefore, there is a need for a low-cost, real-time flood level monitoring system that can continuously measure water levels, identify dangerous conditions and provide immediate warnings to support early disaster response and management.

Proposed Solution:

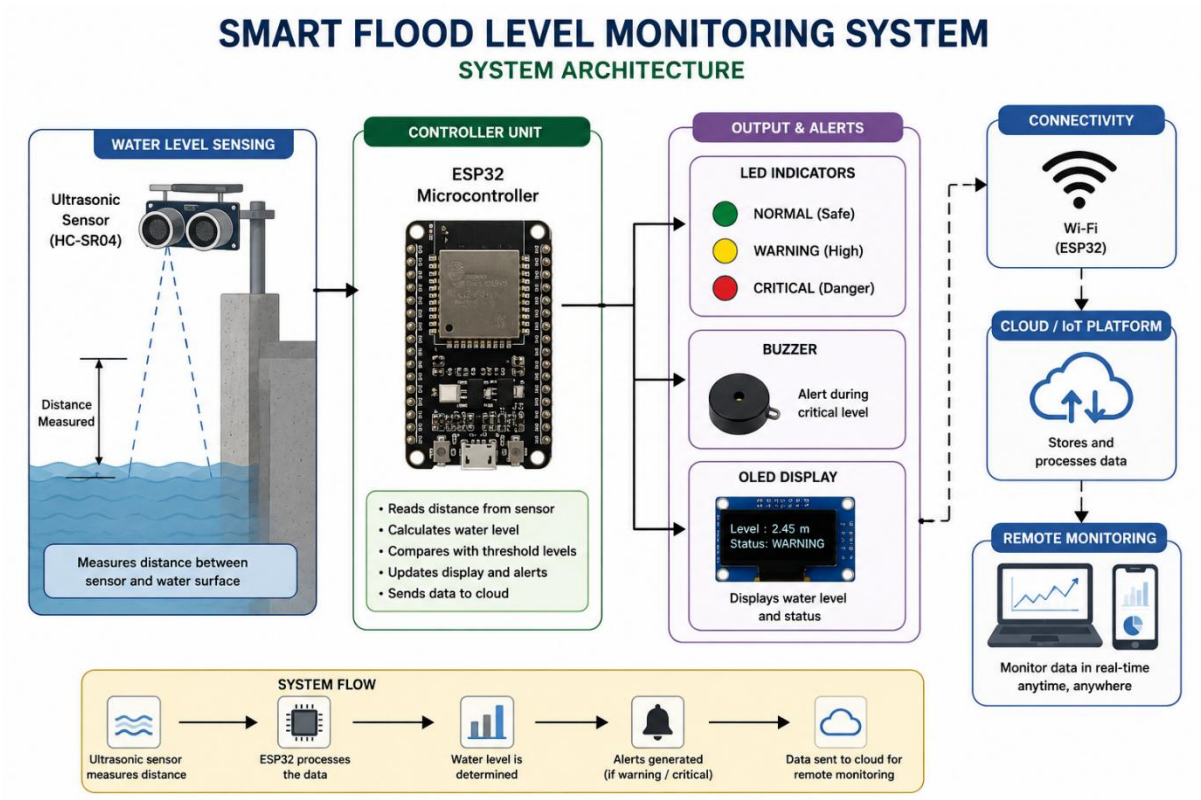
The proposed system uses an ultrasonic sensor and ESP32 microcontroller to continuously monitor the water level in a river, tank, reservoir or drainage channel. The ultrasonic sensor is positioned above the water surface and measures the distance between the sensor and the water.

The ESP32 processes the measured distance and calculates the corresponding water level. Based on predefined threshold values, the system classifies the condition into Normal, Warning and Critical levels.

A green, yellow and red LED can be used to indicate the different water-level conditions. When the water level reaches the critical threshold, a buzzer is activated to provide an immediate warning. The ESP32 can also use its Wi-Fi connectivity to transmit water-level data to an IoT/cloud platform for remote monitoring.

The system provides a low-cost, real-time and automated flood warning solution that can help authorities and users identify rising water levels early and take appropriate disaster management measures.

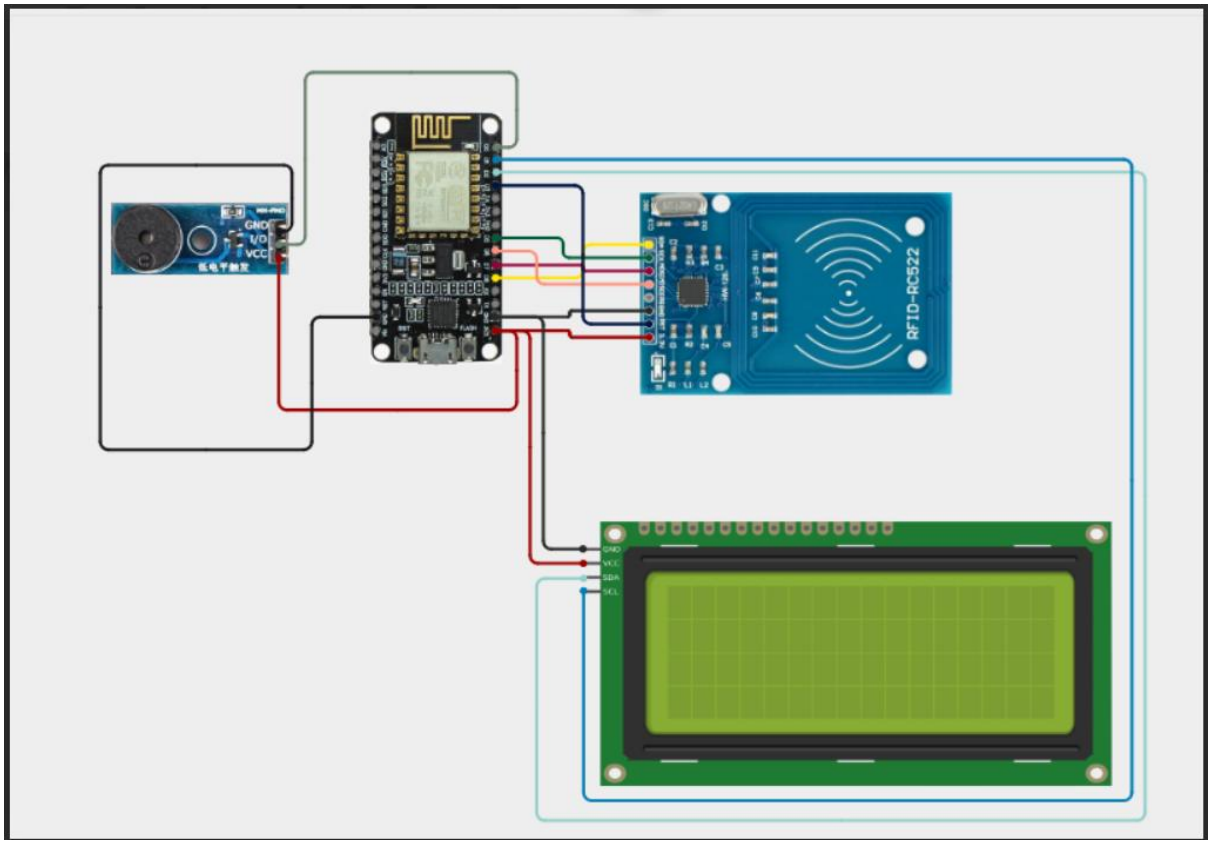
System Architecture:



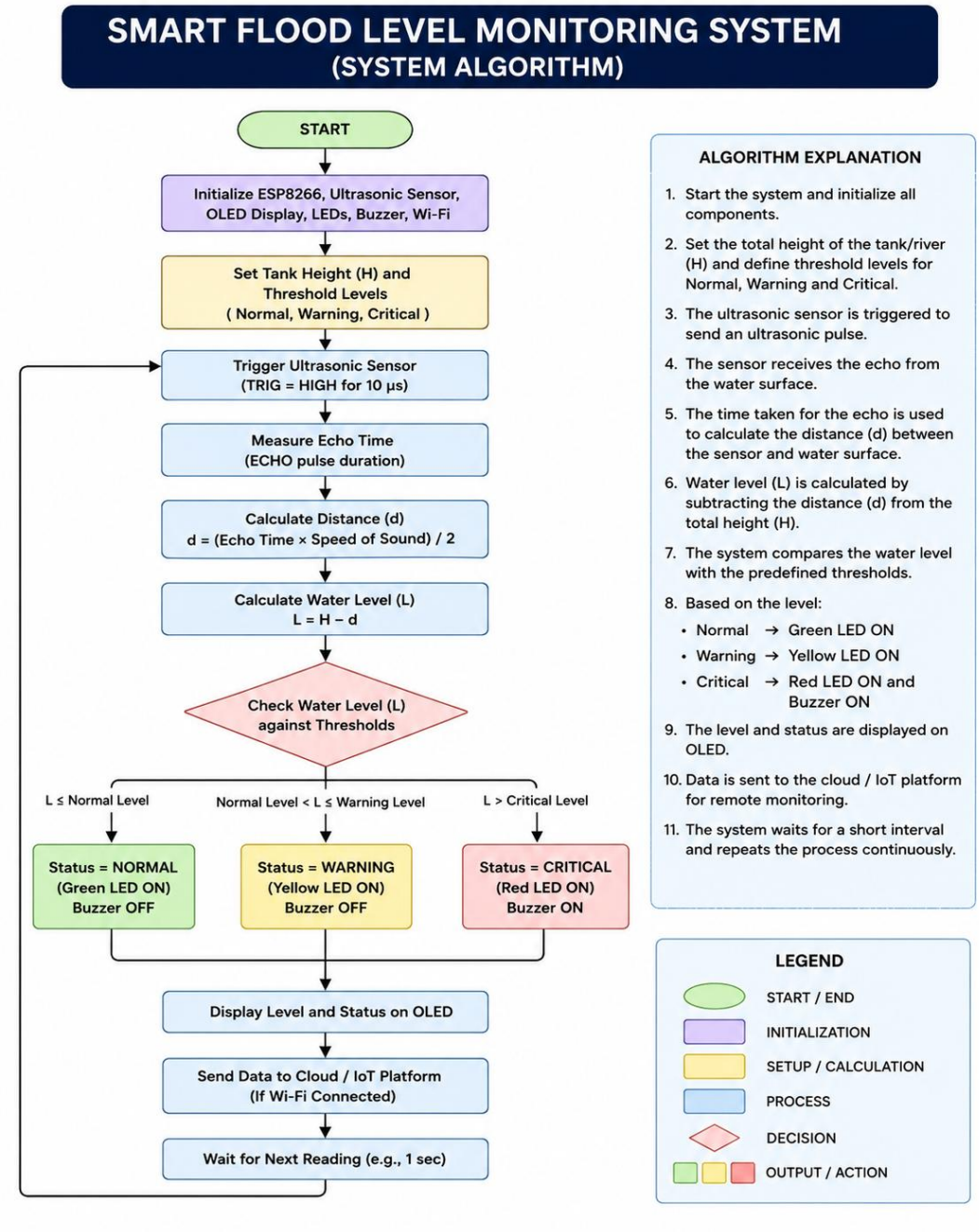
Circuit Table :

Component	Pin/Terminal	ESP8266 Connection	Purpose
Ultrasonic sensor	ECHO	D5	Receives ultrasonic pulse
Ultrasonic sensor	TRIG	D6	Sends ultrasonic pulse
LCD Display	VCC	3.3V	Safe indication
ESP8266	VIN/USB	5V USB	Hazardous indication
Buzzer	Positive (+)	GPIO 14	Alert

Circuit Design:



Algorithm:



Coding:

```
#include <ESP8266WiFi.h>
```

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Adafruit_MQTT.h>
#include <Adafruit_MQTT_Client.h>
#include <math.h>

// ========================================================
// WIFI SETTINGS
// ========================================================

#define WIFI_SSID "Hari"
#define WIFI_PASS "11111111"

// ========================================================
// ADAFRUIT IO SETTINGS
// ========================================================

#define AIO_SERVER     "io.adafruit.com"
#define AIO_SERVERPORT 1883

#define AIO_USERNAME   "Hari_87789"

// IMPORTANT:
// Put your CURRENT Adafruit IO key here.
// Do not use the old exposed key.
#define AIO_KEY        "aio_mTwj83UKedBOZF7ynQuOfANn9JK0"

// ========================================================
// UNIQUE MQTT CLIENT ID
// ========================================================
```

```
#define MQTT_CLIENT_ID "FloodMonitorESP8266"

// ========================================================
// WIFI CLIENT
// ========================================================

WiFiClient client;

// ========================================================
// MQTT CLIENT
// ========================================================

Adafruit_MQTT_Client mqtt(
  &client,
  AIO_SERVER,
  AIO_SERVERPORT,
  MQTT_CLIENT_ID,
  AIO_USERNAME,
  AIO_KEY
);

// ========================================================
// ADAFRUIT IO FEEDS
// ========================================================

Adafruit_MQTT_Publish waterDistance =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/water-distance"
```

```
  );

Adafruit_MQTT_Publish waterLevel =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/water-level"
  );

// =============================================================
// PIN DEFINITIONS
// =============================================================

// HC-SR04
// D5 = GPIO14
#define TRIG_PIN 14

// D6 = GPIO12
#define ECHO_PIN 12

// LED 1
// D7 = GPIO13
#define LED1_PIN 13

// LED 2
// D8 = GPIO15
#define LED2_PIN 15

// LOW ACTIVE BUZZER
// D0 = GPIO16
#define BUZZER_PIN 16
```

```
// ========================================================
// LCD
// ========================================================

// SDA = D2 = GPIO4
#define SDA_PIN 4

// SCL = D1 = GPIO5
#define SCL_PIN 5

LiquidCrystal_I2C lcd(0x27, 16, 2);

// ========================================================
// WATER LEVEL THRESHOLDS
// ========================================================

#define LEVEL1_DISTANCE 20
#define LEVEL2_DISTANCE 10

// ========================================================
// TIMINGS
// ========================================================

// LED1 slow blink
#define LED1_BLINK_TIME 1000

// LED2 fast blink
#define LED2_BLINK_TIME 150
```

```
// Buzzer ON
#define BUZZER_ON_TIME 200

// Buzzer OFF
#define BUZZER_OFF_TIME 800

// Sensor reading interval
#define SENSOR_READ_TIME 500

// Adafruit IO update interval
#define ADAFRUIT_UPDATE_TIME 5000

// MQTT reconnect interval
#define MQTT_RECONNECT_TIME 10000

// LCD update interval
#define LCD_UPDATE_TIME 500

// ============================================================
// TIMER VARIABLES
// ============================================================

unsigned long led1Timer = 0;
unsigned long led2Timer = 0;
unsigned long buzzerTimer = 0;

unsigned long sensorTimer = 0;
unsigned long adafruitTimer = 0;
unsigned long mqttTimer = 0;
unsigned long lcdTimer = 0;
```

```
// ========================================================
// OUTPUT STATES
// ========================================================

bool led1State = false;
bool led2State = false;

// LOW ACTIVE BUZZER
// HIGH = OFF
// LOW = ON

bool buzzerState = HIGH;

// ========================================================
// SENSOR VARIABLES
// ========================================================

int distance = -1;
int level = 0;

// ========================================================
// WIFI CONNECT
// ========================================================

bool connectWiFi()
{
  Serial.println();
  Serial.println("================================");
  Serial.println("CONNECTING TO WIFI");
```

```
Serial.println("================================");

WiFi.mode(WIFI_STA);

WiFi.begin(
  WIFI_SSID,
  WIFI_PASS
);

Serial.print("Connecting");

unsigned long startTime = millis();

while (
  WiFi.status() != WL_CONNECTED &&
  millis() - startTime < 20000
)
{
  delay(500);
  Serial.print(".");
}

Serial.println();

if (WiFi.status() == WL_CONNECTED)
{
  Serial.println("WiFi Connected!");

  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
```

```
    Serial.print("Signal Strength: ");
    Serial.print(WiFi.RSSI());
    Serial.println(" dBm");

    return true;
  }

  Serial.println("WiFi FAILED!");

  return false;
}

// ====================================================
// ADAFRUIT IO CONNECT
// ====================================================

bool connectAdafruitIO()
{
  if (WiFi.status() != WL_CONNECTED)
  {
    return false;
  }

  if (mqtt.connected())
  {
    return true;
  }

  Serial.println();
```

```
  Serial.println("Connecting to Adafruit IO...");

  int8_t ret = mqtt.connect();

  if (ret == 0)
  {
    Serial.println("==================================");
    Serial.println("ADAFRUIT IO CONNECTED!");
    Serial.println("==================================");

    return true;
  }

  Serial.print("Adafruit IO MQTT error: ");
  Serial.println(ret);

  mqtt.disconnect();

  return false;
}

// ========================================================
// ULTRASONIC SENSOR
// ========================================================

int readDistance()
{
  // Trigger LOW

  digitalWrite(
```

```
  TRIG_PIN,
  LOW
);

delayMicroseconds(5);

// Trigger HIGH for 10 us

digitalWrite(
  TRIG_PIN,
  HIGH
);

delayMicroseconds(10);

digitalWrite(
  TRIG_PIN,
  LOW
);

// Read ECHO

unsigned long duration =
  pulseIn(
    ECHO_PIN,
    HIGH,
    30000
  );

// No echo
```

```
  if (duration == 0)
  {
    return -1;
  }

  // Calculate distance

  float distanceCM =
    duration * 0.0343 / 2.0;

  // Reject invalid readings

  if (
    distanceCM < 2 ||
    distanceCM > 400
  )
  {
    return -1;
  }

  // Round to nearest cm

  int roundedDistance =
    (int)round(distanceCM);

  return roundedDistance;
}

// ========================================================
```

```
// DETERMINE WATER LEVEL
// ========================================================

int getWaterLevel(int distanceCM)
{
 if (distanceCM <= LEVEL2_DISTANCE)
 {
  return 2;
 }

 if (distanceCM <= LEVEL1_DISTANCE)
 {
  return 1;
 }

 return 0;
}

// ========================================================
// NORMAL LEVEL
// ========================================================

void normalLevel(unsigned long now)
{
 // LED1 SLOW BLINK

 if (
  now - led1Timer >=
  LED1_BLINK_TIME
 )
```

```
  {
    led1Timer = now;

    led1State = !led1State;

    digitalWrite(
      LED1_PIN,
      led1State
    );
  }

  // LED2 OFF

  digitalWrite(
    LED2_PIN,
    LOW
  );

  led2State = false;

  // BUZZER OFF

  digitalWrite(
    BUZZER_PIN,
    HIGH
  );

  buzzerState = HIGH;
}
```

```
// ========================================================
// LEVEL 1
// ========================================================

void level1Control()
{
 // LED1 ON

 digitalWrite(
  LED1_PIN,
  HIGH
 );

 led1State = true;

 // LED2 OFF

 digitalWrite(
  LED2_PIN,
  LOW
 );

 led2State = false;

 // BUZZER OFF

 digitalWrite(
  BUZZER_PIN,
  HIGH
 );
```

```
  buzzerState = HIGH;
}

// ========================================================
// LEVEL 2
// ========================================================

void level2Control(unsigned long now)
{
  // LED1 ON

  digitalWrite(
    LED1_PIN,
    HIGH
  );

  led1State = true;

  // ======================================================
  // LED2 FAST BLINK
  // ======================================================

  if (
    now - led2Timer >=
    LED2_BLINK_TIME
  )
  {
    led2Timer = now;
```

```
  led2State = !led2State;

  digitalWrite(
    LED2_PIN,
    led2State
  );
}

// ====================================================
// LOW ACTIVE BUZZER
// ====================================================

if (buzzerState == HIGH)
{
  // Currently OFF

  if (
    now - buzzerTimer >=
    BUZZER_OFF_TIME
  )
  {
    buzzerTimer = now;

    buzzerState = LOW;

    // LOW = ON

    digitalWrite(
      BUZZER_PIN,
      LOW
```

```
      );
    }
  }
  else
  {
    // Currently ON

    if (
      now - buzzerTimer >=
      BUZZER_ON_TIME
    )
    {
      buzzerTimer = now;

      buzzerState = HIGH;

      // HIGH = OFF

      digitalWrite(
        BUZZER_PIN,
        HIGH
      );
    }
  }
}

// ================================================
// CONTROL OUTPUTS
// ================================================
```

```
void controlOutputs(
  int currentLevel,
  unsigned long now
)
{
  if (currentLevel == 0)
  {
    normalLevel(now);
  }
  else if (currentLevel == 1)
  {
    level1Control();
  }
  else
  {
    level2Control(now);
  }
}

// ============================================================
// LCD
// ============================================================

void updateLCD()
{
  lcd.clear();

  // First line

  lcd.setCursor(0, 0);
```

```
  lcd.print("Water:");

  lcd.print(distance);

  lcd.print("cm");

  // Second line

  lcd.setCursor(0, 1);

  if (level == 0)
  {
    lcd.print("NORMAL");
  }
  else if (level == 1)
  {
    lcd.print("LEVEL 1 RISING");
  }
  else
  {
    lcd.print("LEVEL 2 DANGER");
  }
}

// ====================================================
// SEND DATA TO ADAFRUIT IO
// ====================================================

void sendAdafruitData()
```

```
{
 if (!mqtt.connected())
 {
  Serial.println(
    "Adafruit IO not connected"
  );

  return;
 }

 // =====================================================
 // WATER DISTANCE
 // =====================================================

 Serial.print(
  "Sending distance: "
 );

 Serial.print(distance);

 Serial.println(" cm");

 bool distanceResult =
  waterDistance.publish(
    distance
  );

 if (distanceResult)
 {
  Serial.println(
```

```
    "Distance sent successfully"
  );
}
else
{
  Serial.println(
    "Distance send FAILED"
  );
}

// ====================================================
// WATER LEVEL
// ====================================================

Serial.print(
  "Sending water level: "
);

Serial.println(level);

bool levelResult =
  waterLevel.publish(
    level
  );

if (levelResult)
{
  Serial.println(
    "Water level sent successfully"
  );
```

```
  }
  else
  {
    Serial.println(
      "Water level send FAILED"
    );
  }

  Serial.println(
    "------------------------------"
  );
}

// ========================================================
// SETUP
// ========================================================

void setup()
{
  Serial.begin(115200);

  delay(1000);

  Serial.println();
  Serial.println("================================");
  Serial.println(" FLOOD LEVEL MONITOR");
  Serial.println("================================");

  // ====================================================
  // PIN MODES
```

```
// ========================================================

pinMode(
  TRIG_PIN,
  OUTPUT
);

pinMode(
  ECHO_PIN,
  INPUT
);

pinMode(
  LED1_PIN,
  OUTPUT
);

pinMode(
  LED2_PIN,
  OUTPUT
);

pinMode(
  BUZZER_PIN,
  OUTPUT
);

// ========================================================
// INITIAL STATES
// ========================================================
```

```
digitalWrite(
  TRIG_PIN,
  LOW
);

digitalWrite(
  LED1_PIN,
  LOW
);

digitalWrite(
  LED2_PIN,
  LOW
);

// LOW ACTIVE BUZZER
// HIGH = OFF

digitalWrite(
  BUZZER_PIN,
  HIGH
);

// ======================================================
// LCD
// ======================================================

Wire.begin(
  SDA_PIN,
```

```
  SCL_PIN
);

lcd.init();

lcd.backlight();

lcd.clear();

lcd.setCursor(0, 0);
lcd.print("FLOOD MONITOR");

lcd.setCursor(0, 1);
lcd.print("Starting...");

delay(2000);

// ================================================
// WIFI
// ================================================

lcd.clear();

lcd.setCursor(0, 0);
lcd.print("WiFi Connecting");

bool wifiOK =
  connectWiFi();

if (wifiOK)
```

```
{
  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("WiFi Connected");

  lcd.setCursor(0, 1);
  lcd.print("AIO Connecting");

  delay(1000);

  // First MQTT attempt

  connectAdafruitIO();
}
else
{
  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("WiFi ERROR");

  lcd.setCursor(0, 1);
  lcd.print("Check WiFi");

  delay(2000);
}

// ================================================
// START SENSOR
```

```
  // ========================================================

  lcd.clear();

  Serial.println();
  Serial.println(
    "SYSTEM READY"
  );

  Serial.println();
}

// ========================================================
// MAIN LOOP
// ========================================================

void loop()
{
  unsigned long now = millis();

  // ========================================================
  // WIFI CHECK
  // ========================================================

  if (
    WiFi.status() != WL_CONNECTED
  )
  {
    Serial.println();
    Serial.println(
```

```
    "WiFi disconnected!"
  );

  connectWiFi();

  // Reset MQTT timer

  mqttTimer = now;
}

// ======================================================
// MQTT RECONNECTION
// ONLY EVERY 10 SECONDS
// ======================================================

if (
  WiFi.status() == WL_CONNECTED &&
  !mqtt.connected()
)
{
  if (
    now - mqttTimer >=
    MQTT_RECONNECT_TIME
  )
  {
    mqttTimer = now;

    connectAdafruitIO();
  }
}
```

```
// ======================================================
// MQTT PROCESSING
// ======================================================

if (mqtt.connected())
{
  mqtt.processPackets(10);
}

// ======================================================
// SENSOR READING
// ======================================================

if (
  now - sensorTimer >=
  SENSOR_READ_TIME
)
{
  sensorTimer = now;

  int newDistance =
    readDistance();

  // ==================================================
  // VALID SENSOR READING
  // ==================================================

  if (newDistance >= 0)
  {
```

```
distance = newDistance;

level =
  getWaterLevel(
    distance
  );

// Control LEDs and buzzer

controlOutputs(
  level,
  now
);

// Serial Monitor

Serial.print(
  "Water Distance: "
);

Serial.print(
  distance
);

Serial.print(
  " cm | Water Level: "
);

Serial.println(
  level
```

```
    );
  }

  // ================================================
  // INVALID SENSOR READING
  // ================================================

  else
  {
    Serial.println(
      "Ultrasonic: NO ECHO"
    );

    // IMPORTANT:
    // Do NOT change distance to 0.
    // Keep previous valid reading.
  }
}

// ================================================
// LCD UPDATE
// ================================================

if (
  distance >= 0 &&
  now - lcdTimer >=
  LCD_UPDATE_TIME
)
{
  lcdTimer = now;
```

```
  updateLCD();
 }

 // ======================================================
 // ADAFRUIT IO UPDATE
 // EVERY 5 SECONDS
 // ======================================================

 if (
  now - adafruitTimer >=
  ADAFRUIT_UPDATE_TIME
 )
 {
  adafruitTimer = now;

  sendAdafruitData();
 }

 // ======================================================
 // SMALL DELAY
 // ======================================================

 delay(10);
}
```

System Working:

- The **Smart Flood Level Monitoring System** uses an **ultrasonic sensor and ESP8266** to continuously monitor the water level. The ultrasonic sensor is placed above the water surface and sends ultrasonic waves toward the water.

- The sensor receives the reflected wave and measures the **echo time**. The ESP8266 uses this time to calculate the distance between the sensor and the water surface. The actual water level is then determined by subtracting the measured distance from the known height of the monitoring area.
- The calculated water level is compared with predefined **Normal, Warning and Critical** thresholds. The corresponding LED indicates the water-level condition. When the water level reaches the critical limit, the **red LED and buzzer are activated** to provide an immediate warning.
- The water level and system status can also be displayed on an **OLED display**. With the ESP8266's Wi-Fi capability, the measured data can be transmitted to a cloud or IoT platform for remote monitoring.
- The system continuously repeats the measurement and alert process, providing **real-time flood-level monitoring and early warning** for disaster management applications.

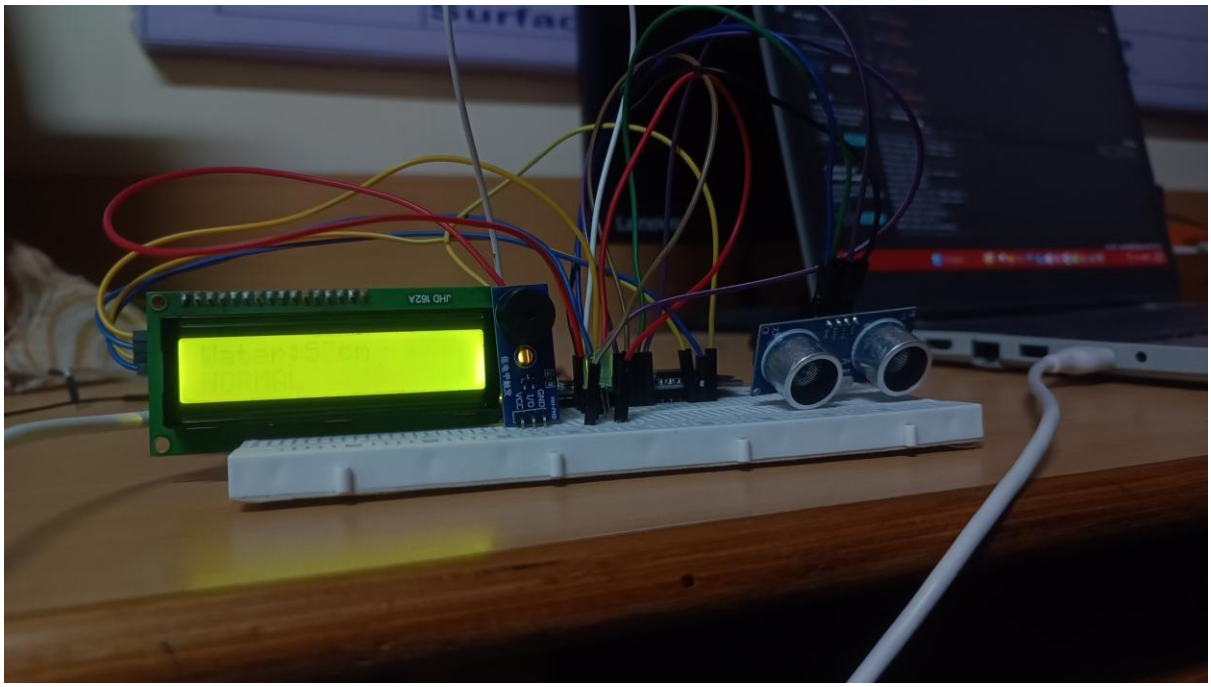
Bill Of Material:

S. no	Component	Specification	Quantity	Cost
1	ESP8266	Node MCU ESP8266	1	Rs. 450
2	Ultrasonic sensor	HC-SR04 Ultrasonic sensor	1	Rs. 70
3	OLED Display	0.96" I²C, SSD1306, 128×64	1	Rs. 150
5	Breadboard	Standard 830-point	1	Rs. 100
6	Jumper Wire	Male-Male/Male-Female	1 set	Rs. 50
			Total	Rs. 820

Prototype Implementation:

- The prototype was implemented using an **ESP8266 NodeMCU, HC-SR04 ultrasonic sensor, OLED display, three LEDs and a buzzer**. The ultrasonic sensor was positioned above a water container to continuously measure the distance between the sensor and the water surface.

- The ESP8266 receives the echo signal from the ultrasonic sensor and calculates the water level based on the measured distance. The calculated level is compared with predefined **Normal, Warning and Critical** thresholds.
- A **green LED** indicates a normal water level, a **yellow LED** indicates a warning condition, and a **red LED with buzzer** provides an alert when the water level reaches the critical limit. The current water level and status are also displayed on the OLED screen.
- The ESP8266's built-in Wi-Fi capability can be used to transmit the measured water-level data to a cloud or IoT platform for remote monitoring. The prototype continuously updates the readings, providing a simple and low-cost solution for **real-time flood-level monitoring and early warning**.



Results and Performance:

The developed **Smart Flood Level Monitoring System** successfully monitored the water level using the **HC-SR04 ultrasonic sensor and ESP8266**. The sensor continuously measured the distance between the sensor and the water surface, and the ESP8266 calculated the corresponding water level.

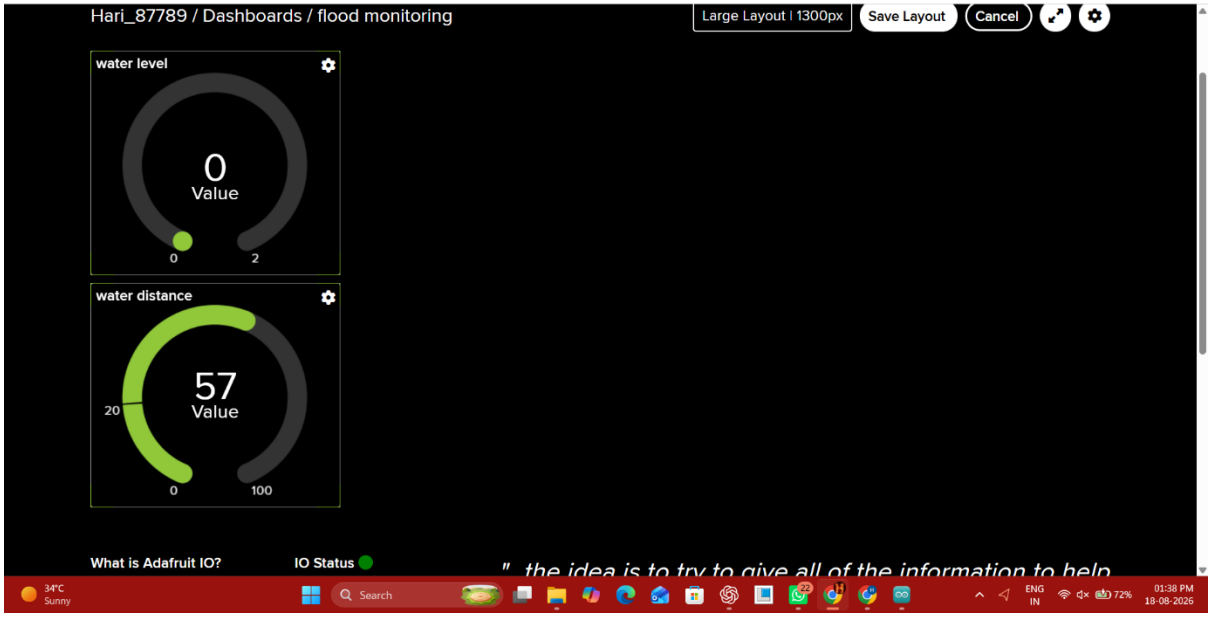
The system successfully classified the water level into **Normal, Warning and Critical** conditions based on predefined threshold values. The green, yellow and red LEDs provided clear visual indications, while the buzzer generated an immediate alert during critical water levels. The OLED display also provided real-time information about the measured water level and current status.

Performance Analysis

Parameter	**Result**
Water-Level Measurement	Successfully monitored using ultrasonic sensor
Level Calculation	Real-time calculation by ESP8266
Normal-Level Detection	Green LED activated
Warning-Level Detection	Yellow LED activated
Critical-Level Detection	Red LED + buzzer activated
Local Monitoring	OLED display
Response	Continuous and real-time
Remote Monitoring	Wi-Fi/IoT integration supported

Overall, the prototype demonstrated **reliable real-time water-level monitoring and early warning capability**. The system is low-cost, easy to implement and suitable for small-scale flood monitoring applications. Its performance can be further improved by using waterproof ultrasonic sensors, solar power, multiple sensing points and cloud-based alerts for large-scale disaster management.

.


Hari_87789 / Dashboards / flood monitoring
Large Layout | 1300px
Save Layout
Cancel
water level
0
Value
0
2
water distance
57
Value
20
0
100
What is Adafruit IO?
IO Status
" the idea is to try to give all of the information to help
34°C
Sunny
Search
ENG
IN
72%
01:38 PM
18-08-2026